ELARA: Energy-Efficient and Latency-Aware Microservice Orchestration in Space Computing Power Networks

Anonymous Authors

***Abstract*—The rapid advancement of the Space Computing Power Network (SCPN), comprising numerous low Earth orbit (LEO) satellites, has enabled the execution of complex in-orbit observation and computation tasks. To facilitate this, microservice architecture decomposes large, monolithic remote sensing and image processing applications into independent, schedulable, and reusable services deployed in a distributed manner across LEO satellites. However, integrating these capabilities presents significant challenges arising from the limited computing resources, constrained power provisions of individual satellites, and the dynamic nature of space network topology and channel conditions. This paper addresses these challenges by proposing an energy-aware service routing algorithm for microservices within the SCPN. The algorithm aims to minimize the energy consumed by the procedures of processing and transmitting service data in both computation and communication. By modeling the service routing problem as a Directed Steiner Tree (DST) and implementing heuristic-based solutions, our algorithm effectively responds to the diverse service patterns requested by terrestrial clients. Comprehensive experimental results demonstrate that our approach significantly outperforms existing methods in terms of energy efficiency, robustness, stability, and adaptability.**

***Index Terms*—Energy, Microservice, Routing, Space Network**

I. Introduction

Recent advances in low Earth orbit (LEO) mega-constellations are transforming satellites from bent-pipe communication relays into distributed computing nodes in space. These large-scale constellations, comprising hundreds to thousands of satellites connected by inter-satellite links (ISLs) and equipped with onboard processors and storage resources, enable global connectivity and near-data processing for time-sensitive applications [1]–[3]. This evolution is particularly critical for Earth observation, disaster response, environmental monitoring, and remote sensing analytics, where massive volumes of images and multi-modal data are continuously generated in orbit [4]–[6]. Directly downloading such raw data to terrestrial data centers for analysising will consume scarce satellite-to-ground bandwidth and significantly delays mission response, especially under limited contact windows and unfavorable channel conditions [7], [8]. As a result, space computing power networks (SCPNs), which pool sensing, communication, and computing resources across LEO satellites, have emerged as a promising infrastructure for onboard intelligent services [9], [10].

Promising but challenging, running these complex monolithic applications on a single LEO satellite remains infeasible due to stringent constraints on computing, storage, energy privisions, and thermal dissipation [11], [12]. Microservice architecture provides an effective solution by decomposing these remote sensing and AI applications into fine-grained, loosely coupled, and reusable microservices, each encapsulating a distinct functional module [13]–[16]. By packaging them into lightweight containers and deploying them across interconnected satellites, this architecture alleviates individual satellite resource limitations and enables onboard cooperative execution of complex applications. Furthermore, upon detecting hotspot microservices, the system can dynamically scale their replicas and adjust deployment strategies to ensure robust and efficient service provisioning.

Typically, a single request initiated by terrestial user involves a subset of microservices. Due to its arbitrary arrival time, the intermediate data transmission process can be affected by dynamic and time-varying ISL states. Moreover, the harware heterogeneity and limited onboard energy provision further complicate the scheduling decisions at each microservice execution stage.

To enable energy-efficient and low-latency service provision in SCPNs, there are several major challenges must be addressed. First, the satellite network topology is predictable but highly time-varying. Intra-plane ISLs are relatively stable, whereas cross-plane ISLs are sensitive to orbital geometry and change frequently across slots. Consequently, routing intermediate data along a single static path often suffers from link disruptions, excessive detours, or degraded transmission rates. Second, serving satellite selection and ISL routing are tightly coupled. Selecting a nearby replica may reduce transmission delay but lead to long computation queues on overloaded satellites, while choosing an idle satellite can incur high multi-hop communication costs. This coupling is further complicated by satellite energy constraints, as both onboard computation and laser-based inter-satellite transmission consume scarce power resources [17]–[19]. Third, microservice replica deployment cannot remain static. As request distributions, satellite workloads, and network topology evolve, previously optimal placements quickly become inefficient. Nevertheless, frequent redeployment is costly because of limited ISL bandwidth for image transfers and bounded storage capacity on each satellite.

Existing studies have extnsively investigated satellite edge computing, in-orbit computation, service placement, and NFV-enabled service chaining in the context of SCPN [7], [8], [20]–[24]. However, most of them fail to fully capture the continuous execution of microservice chains and ignore the

joint optimization of service routing and replica deployment under time-varying communication conditions and heterogeneous onboard hareware environments. In particular, many satellite-oriented service and function execution models rely on quasi-static constellation snapshots, implicitly assuming that each task can be completed within a single period during which link states remain unchanged. While this abstraction simplifies modeling and algorithm design, it deviates from practical in-orbit service execution, where data routing, computation, and scheduling could frequently span multiple time slots [25]. Although terrestrial microservice routing studies have explored the joint optimization of service deployment and request routing [26], [27], their assumptions of stable networks, abundant power supply, and fixed edge infrastructure are not directly applicable to the SCPNs.

A. Contributions

To address the aforementioned challenges, we propose ELARA, an energy-efficient and low-latency microservice orchestration framework for SCPNs. ELARA jointly optimizes routing and deployment decisions for distributed microservice-based applications in spatio-temporally dynamic SCPN environments. Specifically, upon a request arrives at an arbitrary continuous time, ELARA divides the entire execution process into multiple sequential stages. For each execution stage, after completing the current service on a serving satellite, ELARA first selects the next computing node from the subset of satellites hosting replicas of the subsequent microservice, while considering the volume of intermediate data, real-time onboard computing capacities, and relay ISL states. ELARA then executes a service routing algorithm based on the source and destination satellites, available bandwidths, and congestion levels of ISLs, generating a set of parallel routing paths to accelerate data transmission and enhance robustness against link disruptions caused by the time-varying constellation topology. Finally, ELARA periodically adjusts microservice deployments according to historical service invocation patterns and the real-time computing and communication conditions of the SCPN. Built upon this design, ELARA introduces a continuous-time cross-slot service provisioning mechanism along with an adaptive routing-and-redeployment strategy. This enables the system to jointly optimize end-to-end latency, energy consumption, and deadline satisfaction under dynamic constellation topologies and constrained onboard resources.

The main contributions of this paper are summarized as follows:

1) We identify and formalize a new microservice-chain execution paradigm for SCPNs, where stateless service replicas are distributed across LEO satellites and each request may arrive at an arbitrary time. This formulation captures the coupled impact of serving satellite selection, inter-satellite routing, onboard computation, and replica deployment on request-level service quality.
2) We develop a dynamic SCPN system model that characterizes Walker-Delta constellation dynamics, time-varying ISL availability, distance-dependent link capacity, communication latency and energy, and onboard computation latency and energy. This model enables cross-slot service execution analysis by explicitly accounting for queuing, transmission, propagation, waiting, and execution costs under evolving network and computing conditions.
3) We formulate the ELARA service routing problem as an online energy- and latency-aware optimization problem over three coupled decisions: serving satellite selection, cross-slot multi-hop routing, and adaptive replica redeployment. The formulation incorporates link-capacity, replica-count, and onboard storage constraints, thereby capturing the practical resource limitations of SCPNs.
4) We design ELARA to coordinate request-level routing with periodic replica redeployment by exploiting real-time satellite computing states and time-varying ISL topology. Extensive simulations under dynamic LEO constellations and diverse service-chain workloads demonstrate that ELARA reduces end-to-end latency, energy consumption, and deadline violations compared with representative baselines.

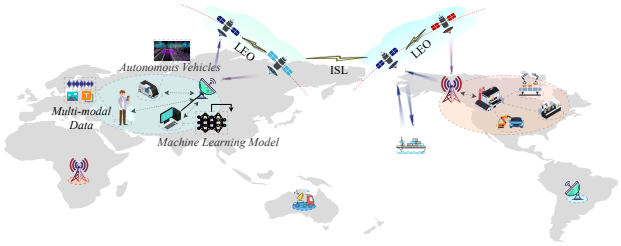


Fig. 1: Space and terrestrial network topology.

The remainder of this paper is organized as follows. Section II reviews the related works. Section III presents the system models. The problem formulation is introduced in Section IV. Section V details the proposed heuristic algorithms. Section VI shows and analyses the Simulation results. Section VII draws the conclusion.

II. Related Work

Recent advancements in satellite technology have significantly promoted its computing and transmitting capabilities, enabling satellites to undertake some elementary tasks [9]. The construction of ISLs, in particular, enables interconnected satellites to form the SCPN with pooled computing resources, supporting intricate missions with enhanced performance demands [7], [11], [28] and facilitating a global response to requests from anywhere at any time [29], [30]. In addition, careful energy management is necessary for satellites to realize sustainable development [17], [18].

Considering the discrete distributed computing nodes (as satellites) with limited computing power in the SCPN, researchers have migrated the monolithic applications (such as 3D reconstruction and inference model), which are traditionally deployed on ground data centers, to space networks by decomposing them into microservices, which are characterized by being independent, scalable, reusable, and stateless [12],

[15], [16]. These microservices can be scaled and deployed on the satellites through platform-independent containerization technology, which encapsulates these isolated microservices into containers that are transparent to the installed computing platform on satellites [10], [13].

Extensive research has been conducted on service routing algorithms, aiming at the problems of node selection and routine planning. Fu et al. [31] proposed a dynamic programming approach to drive the optimal policy on energy allocation for a single satellite in response to user requests. It fails to consider the multi-satellite cases. The work [20] designed a joint edge server placement and service deployment model for SAGIN, aiming to minimize the average response latency and energy cost. The work [17] developed a series of heuristic algorithms to prolong the serving life of satellites by switching part of routers installed on satellites to sleep mode, and resolving the energy-efficient routing policies. Efficient but shortsighted, these proposed algorithms cannot adapt to diverse routing needs, for example, the treed topology raised by requested missions, which demands integrated computing and communication routing as well as multi-satellite collaboration.

III. System Model

We consider a space computing power network (SCPN) composed of a Walker-Delta LEO constellation, where each satellite is equipped with onboard computing resources and laser inter-satellite links (ISLs). User tasks are represented as ordered microservice chains. Each microservice can have multiple stateless replicas deployed on different satellites, and a request may therefore be executed collaboratively by multiple satellites.

A. LEO Constellation and Time-slotted Topology

A typical Walker-Delta constellation is parameterized by $\mathcal{W} = (A/P/G, H, I)$, where A is the total number of satellites, P is the number of orbital planes, G is the inter-plane phasing factor, H is the orbital altitude, and I is the inclination. Consequently, each orbital plane accommodates $N = A/P$ satellites. The set of satellite nodes in the constellation is formally defined as $\mathcal{V} = \{v_{p,n} \mid p = 1, \ldots, P,\ n = 1, \ldots, N\}$, where p denotes the orbital plane index, n denotes the intra-plane position index, To capture the high dynamism of the satellite network, we discretize the constellation's operational period into a set of time slots, as $\mathcal{T} = \{0, 1, \ldots, T-1\}$, with a uniform slot duration Δ. For any continuous-time instant τ, the corresponding time slot index is mapped as, $s(\tau) = \lfloor \tau/\Delta \rfloor \bmod T$. Adopting a snapshot-based modeling approach, we assume that the network state, including satellite positions, ISL availability and capacity, remains quasi-static within a single time slot, and only evolves at the slot boundaries governed by orbital mechanics. Thus, the time-varying network topology during slot $t \in \mathcal{T}$ is formally defined as a dynamic and time-dependent graph $\mathcal{G}(t) = \big(\mathcal{V}, \mathcal{E}(t)\big)$, where $\mathcal{E}(t)$ represents the set of feasible ISLs at slot t.

B. ISL Communication Model

We assume that each satellite is equipped with four laser optical terminals: two positioned along the roll axis for intra-plane ISLs, and two along the pitch axis for cross-plane ISLs.

a) ISL Connectivity Constraints: Each satellite $v_{p,n}$ maintains two intra-plane ISLs with its adjacent neighbors v_{p,n^-} and v_{p,n^+}, where $n^{\pm} = ((n \pm 1) \bmod N) + 1$. Because satellites in the same plane share identical angular velocities, their relative distances remain time-invariant, yielding highly stable communication links.

Conversely, cross-plane ISLs are inherently dynamic. Constrained by the physical and kinematic limits of onboard laser communication terminals (LCTs), the existence of a cross-plane link $e = (u, v)$ at slot t is jointly dictated by line-of-sight (LoS) occlusion, mechanical tracking thresholds, and polar exclusions, formulated as:

$$e \in \mathcal{E}(t) \iff \begin{cases} d_{u,v}(t) \le d_{\max}, \\ \theta_{u,v}(t) \le \theta_{\max}, \\ \max\big(|\varphi_u(t)|, |\varphi_v(t)|\big) \le \varphi_{\max}. \end{cases} \tag{1}$$

In (1), the LoS distance $d_{u,v}(t)$ and relative tracking angle $\theta_{u,v}(t)$ are bounded by the terrestrial occlusion limit $d_{\max}$ and the hardware threshold $\theta_{\max}$, respectively. Furthermore, the third constraint enforces a polar exclusion zone at latitude boundary $\varphi_{\max}$ (e.g., 70°) through proactively tearing down the cross-plane links at high latitudes, which effectively circumvents the severe pointing dynamics and extreme Doppler shifts inherent to polar regions.

b) ISL Capacity Model: Unlike ground networks with stable capacities, the transmission rate of laser ISLs is distance-dependent due to geometric propagation loss. Based on the Free-Space Optical (FSO) communication model, the received optical power $P_e^{\text{rx}}(t)$ at satellite v transmitted from satellite u is formalized as:

$$P_e^{\text{rx}}(t) = P^{\text{tx}} G^{\text{tx}} G^{\text{rx}} \eta_{\text{tx}} \eta_{\text{rx}} \left(\frac{\lambda}{4\pi d_{u,v}(t)} \right)^2, \tag{2}$$

where P^{tx} is the nominal laser transmit power, G^{tx} and G^{rx} represent the optical gains of the transmitter and receiver apertures respectively, λ is the laser wavelength, and $\eta_{\text{tx}}, \eta_{\text{rx}}$ denote the optical efficiency factors. The real-time Signal-to-Noise Ratio (SNR) of the link e at time slot t is given as:

$$\gamma_e(t) = \frac{\left(R_{\text{pd}} \cdot P_e^{\text{rx}}(t)\right)^2}{\sigma_{\text{sh}}^2 + \sigma_{\text{th}}^2}, \tag{3}$$

where R_{pd} is the photodiode responsivity, while σ_{sh}^2 and σ_{th}^2 denote the shot and thermal noise variances, respectively. According to the Shannon-Hartley theorem, the achievable transmission capacity $R_e^0(t)$ is constrained by:

$$R_e^0(t) = \mathrm{B}_e \log_2\left(1 + \gamma_e(t)\right), \tag{4}$$

where B_e is the modulation bandwidth.

c) ISL Delay Model: The nominal rate $R_e^0(t)$ in (4) could be degraded in practice by transient channel fading and traffic contention. To capture these factors, we introduce a scaling factor $\eta_e(t) \in (0,1]$ to formulate the effective data rate as $R_e(t) = \eta_e(t) R_e^0(t)$. When transmitting a data block of size B over link $e = (u, v)$ during slot t, the aggregate link-level communication delay comprises queuing, transmission, and propagation delays, formulated as:

$$T_e^{\text{cm}}(B,t) = \nu_e^{\text{qe}}(t) + \nu_e^{\text{tr}}(t) + \nu_e^{\text{pr}}(t). \tag{5}$$

Here, $\nu_e^{\text{qe}}(t)$ denotes the queuing delay, which can be dynamically estimated via lightweight queue-length probing. The transmission delay is given by $\nu_e^{\text{tr}}(t) = B/R_e(t)$, and the propagation delay is $\nu_e^{\text{pr}}(t) = d_e(t)/c$, with c being the speed of light in vacuum.

C. Onboard Computing Model

Let F_v^0 denote the nominal computing capacity of satellite v. In practice, the available computational resources are often constrained by background payload tasks, operating system overhead, and thermal throttling. To capture these hardware and software limitations, we introduce a computational efficiency factor $\xi_v(t) \in (0,1]$ and define the effective dynamic computing capacity as $F_v(t) = \xi_v(t) F_v^0$.

Let $\nu_v^{\text{qe}}(t)$ denote the queuing delay before scheduling microservice m is executed on satellite v, which increases with the pending task queue length at slot t. The total computation delay is modeled as:

$$T_m^{\text{cp}}(v,t) = \nu_v^{\text{qe}}(t) + \nu_v^{\text{cp}}(t), \tag{6}$$

where $\nu_v^{\text{cp}}(t) = C_m / F_v(t)$ represents the actual execution delay, and C_m is the number of CPU cycles required by microservice m. Correspondingly, the computational energy consumption is defined as:

$$E_m^{\text{cp}}(v,t) = P_v^{\text{cp}} \frac{C_m}{F_v(t)}, \tag{7}$$

where P_v^{cp} is the operational computing power of the satellite.

D. LEO Microservice and Service Request Models

To facilitate flexible application management within the SCPN, complex applications are decoupled into a set of fine-grained, loosely coupled microservices, denoted by $\mathcal{M} = \{1, 2, \ldots, M\}$. Each microservice $m \in \mathcal{M}$ is characterized by its computational demand C_m (in CPU cycles), container image size I_m, and storage requirement S_m.

We define a binary indication variable $x_{m,v}(t) \in \{0,1\}$, which equals to 1 if m is deployed on v during slot t. Accordingly, the subset of satellite nodes hosting a valid replica of microservice m at slot t is denoted as $\mathcal{R}_m(t) = \{v \in \mathcal{V} \mid x_{m,v}(t) = 1\}$. To ensure service availability while preventing excessive resource contention, the total number of deployed replicas for any microservice should be bounded within predefined thresholds, $r_m^{\min}$ and $r_m^{\max}$:

$$r_m^{\min} \le \sum_{v \in \mathcal{V}} x_{m,v}(t) \le r_m^{\max}, \quad \forall m \in \mathcal{M}. \tag{8}$$

Furthermore, the aggregate storage footprint of all replicas hosted on a single satellite must not exceed its onboard storage capacity $S_v^{\max}$:

$$\sum_{m \in \mathcal{M}} x_{m,v}(t) S_m \le S_v^{\max}, \quad \forall v \in \mathcal{V}. \tag{9}$$

Let $r = \langle v_s, m_1, m_2, \ldots, m_L, v_d, \tau_{\text{in}} \rangle$ denote an incoming request with a deadline D_r. Here, v_s and v_d are the source and destination satellites, m_i is the i-th microservice in the sequential execution chain, and τ_{in} is the continuous arrival time. Let $B_{r,i}$ be the data volume that should be transmitted before invoking m_i, while $B_{r,L+1}$ denotes the final output routed to v_d.

a) Cross-slot Service Routing Latency and Energy Model: Since requests may arrive at arbitrary instants while ISL topology and rates change at slot boundaries, ELARA tracks each service chain in continuous time. Before executing m_i, the input data $B_{r,i}$ stored at satellite $u_{r,i}$ should be routed to a satellite hosting a valid replica of m_i. We introduce a binary decision variable $y_{r,i,v} \in \{0,1\}$ for node selection, which equals 1 if m_i is scheduled to satellite v. Let $v_{r,i}$ be the selected satellite with $y_{r,i,v_{r,i}} = 1$. For the i-th service routing operation, the source and destination satellites are denoted by $u_{r,i}$ and $v_{r,i}$, respectively.

Since the endpoints may be non-adjacent and transmission may start near a slot boundary, one routing operation can span $H \ge 1$ phases over time-varying multi-hop routes. For phase h, let $\tau_{r,i}^h$ be its start time, $t_h = s(\tau_{r,i}^h)$ the active topology slot, $\bar{\Delta}_h$ the remaining slot time, and $B_{r,i}^h$ the residual data, with $B_{r,i}^1 = B_{r,i}$. Let $\mathcal{P}_{r,i}^h$ be the feasible and active end-to-end path set from $u_{r,i}$ to $v_{r,i}$ in slot t_h, where each path $p \in \mathcal{P}_{r,i}^h$ is an ordered sequence of one or more ISLs, i.e., $p = (e_1, \ldots, e_{|p|})$, and different path may partially share the same ISLs. If D_h bits are transmitted, ELARA splits them across multiple such paths, and path p receives fraction $\lambda_p^h \in [0,1]$:

$$d_p^h = \lambda_p^h D_h, \qquad \sum_{p \in \mathcal{P}_{r,i}^h} \lambda_p^h = 1, \qquad 0 < D_h \le B_{r,i}^h. \tag{10}$$

The aggregate load on shared ISL $e \in p$ is formulated by:

$$f_e^h = \sum_{p \in \mathcal{P}_{r,i}^h : e \in p} d_p^h, \qquad 0 \le f_e^h \le R_e(t_h) \bar{\Delta}_h. \tag{11}$$

For a multi-hop path p, the subflow completion time follows the link-delay model in (5):

$$\delta_p^h = \sum_{e \in p} T_e^{\text{cm}}(d_p^h, t_h), \qquad \forall p \in \mathcal{P}_{r,i}^h. \tag{12}$$

Since the selected paths transmit in parallel, the effective phase duration is determined by the slowest active path:

$$\Delta_h^{\text{ru}} = \max_{p \in \mathcal{P}_{r,i}^h} \delta_p^h, \quad \text{and,} \quad \Delta_h^{\text{ru}} \le \bar{\Delta}_h. \tag{13}$$

The phase energy is the sum of transmission energy over all traffic-carrying ISLs:

$$E_{r,i}^{h,\text{ru}} = \sum_{e \in \mathcal{E}(t_h)} P_e^{\text{tx}} \frac{f_e^h}{R_e(t_h)}. \tag{14}$$

After phase h, the residual data and decision epoch are updated as:

$$B_{r,i}^{h+1} = B_{r,i}^{h} - D_h, \qquad \tau_{r,i}^{h+1} = \tau_{r,i}^{h} + \Delta_h^{\text{ru}}. \tag{15}$$

If $B_{r,i}^{h+1} > 0$, the next phase is solved under the updated topology, path set, and link rates. For $H_{r,i}$ phases, the routing latency and energy from $u_{r,i}$ to $v_{r,i}$ are:

$$T_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} \Delta_h^{\text{ru}}, \qquad E_{r,i}^{\text{ru}} = \sum_{h=1}^{H_{r,i}} E_{r,i}^{h,\text{ru}}, \tag{16}$$

The selected replica receives the data at $\tau_{r,i}^{\text{arr}} = \tau_{r,i} + T_{r,i}^{\text{ru}}$. The selected satellite then executes m_i, whose computation delay and energy are given by (6) and (7), respectively.

The cumulative execution latency and energy of request r are given by:

$$T_r^{\text{e2e}} = \sum_{i=1}^{L+1} T_{r,i}^{\text{ru}} + \sum_{i=1}^{L} T_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{17}$$

$$E_r^{\text{tot}} = \sum_{i=1}^{L+1} E_{r,i}^{\text{ru}} + \sum_{i=1}^{L} E_{m_i}^{\text{cp}}\big(v_{r,i}, s(\tau_{r,i}^{\text{arr}})\big), \tag{18}$$

where $i = L+1$ denotes the final output delivery to v_d.

b) Service Replica Migration Model: Let $\mathbf{X}(t) = [x_{m,v}(t)]$ represent the microservice replica deployment matrix at slot t. ElARA periodically updates the deployment matrix every few time slots by selecting an action $a_{m_i} \in \{\text{add}, \text{remove}, \text{move}, \text{no-op}\}$ for each m_i in a microservice subset $\mathcal{M}'(t) \subset \mathcal{M}$ at time slot t. Then we have:

$$\mathbf{X}(t+1) = \Phi(\mathbf{X}(t), \mathcal{A}_t), \tag{19}$$

where $\Phi(\cdot)$ denotes the deployment transition induced by the selected action set $\mathcal{A}_t = \{a_{m_i} | \forall m_i \in \mathcal{M}'(t)\}$. All redeployment actions should preserve the replica-count and storage constraints in (8) and (9).

IV. Problem Formulation of ELARA

We formulate ELARA as an online energy- and latency-aware service routing problem over a dynamic SCPN. The objective is to simultaneously minimize the end-to-end execution latency and total energy consumption of service chains, including both communication and computation costs, while accounting for the overhead of replica redeployment.

A. Problem Formulation

For each incoming request, ELARA first selects the serving satellite v for each microservice m_i, which satisfies:

$$\sum_{v \in \mathcal{V}} y_{r,i,v} = 1, \quad y_{r,i,v} \le x_{m_i,v}\big(s(\tau_{r,i})\big), \quad \forall r, i, v. \tag{20}$$

Let $v_{r,i}$ denote the selected satellite satisfying $y_{r,i,v_{r,i}} = 1$. ELARA then makes the cross-slot routing policy for service-routing segment i:

$$\Pi_{r,i}^{\text{ru}} = \{D_h, \lambda_p^h, f_e^h, \Delta_h^{\text{ru}}\}_{h=1}^{H_{r,i}}, \tag{21}$$

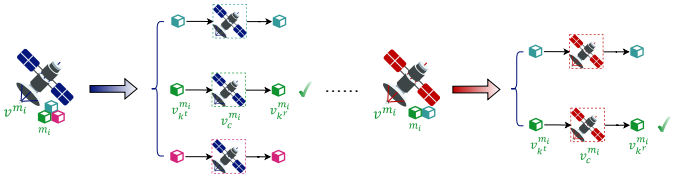


Fig. 2: Abstract the computation process from colocated microservice and satellite deployment.

which follows the multi-hop routing model in Sec. III-D. Additionally, ELARA periodically updates the deployment matrix by applying $\mathcal{A}(t), \forall t \in \mathcal{T}'$, where $\mathcal{T}' \subset \mathcal{T}$.

Following (17) and (18), given an online request r, the request-level optimization objective is defined as:

$$\begin{aligned} \mathscr{P}: \quad & \underset{y_{r,v}, \Pi_r^{\text{ru}}, \mathcal{A}}{\text{minimize}} \quad J_r = \alpha T_r^{\text{e2e}} + \beta E_r^{\text{tot}} + \rho \mathbb{I}_r^{\text{fail}} \\ \text{subject to} \quad & (20), (21), && \text{(C1)} \\ & (10), (11), (13), (15), && \text{(C2)} \\ & (8), (9), (19), && \text{(C3)} \end{aligned}$$

where α, β, and ρ are non-negative weights, and $\mathbb{I}_r^{\text{fail}}$ indicates whether request r violates its deadline. Constraint C1 enforces serving satellite selection by assigning each microservice stage to exactly one satellite hosting a valid replica, and defining the corresponding routing policy for each service routing segment. Constraint C2 ensures feasible cross-slot multi-hop routing under time-varying topology by enforcing path splitting, shared ISL capacity, phase duration, and residual data update constraints. Constraint C3 restricts adaptive replica redeployment to states that satisfy both replica-count and onboard storage constraints after each migration action.

B. Problem Hardness

Problem $\mathscr{P}$ is NP-hard. Even in a single-slot static case with fixed deployment and fixed link and computing capacities, $\mathscr{P}$ requires selecting replicas for a service chain and connecting them through capacity-constrained paths. This restricted case contains the placement of service function chains and constrained routeing as special cases and is therefore NP-hard. The full problem further couples these decisions with cross-slot routeing and adaptive replica redeployment, motivating ELARA's online decomposition.

V. Energy-aware Service Routing Algorithm for Microservices in the SCPN

In this section, we present our approach to addressing the proposed problem **P** as outlined in Section IV. We first detail our augmented satellite-microservice connected sub-graph construction algorithm, and then elaborate on the design of our DST-based service routing algorithm tailored to optimize energy consumption in the SCPN.

A. Augmented Satellite-Microservice Sub-Graph Construction

For each request originating from a terrestrial station, there is a DAG $\mathcal{G}_f^m$, which outlines the service execution sequence.

Algorithm 1: Augmented Satellite-Microservice Connected Subgraph Construction Algorithm.

Initialize augmented directed graph $\mathcal{G}_z$ with vertex set $\mathcal{V}_z \leftarrow \varnothing$ and edge set $\mathcal{E}_z \leftarrow \varnothing$;
$\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \{v^{src}, v^{dst_1}, \ldots, v^{dst_\rho}\}$;
for m_i *in* $\mathcal{M}_f$ **do**
  for v_k *in* $\mathcal{V}_{m_i}$ **do**
    $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \{v_{k^r}^{m_i}, v_{k^c}^{m_i}, v_{k^t}^{m_i}\}$;
    $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \{(v_{k^r}^{m_i} \rightarrow v_{k^c}^{m_i}), (v_{k^c}^{m_i} \rightarrow v_{k^t}^{m_i})\}$;
    Set $\mathbf{w}(v_{k^r}^{m_i} \rightarrow v_{k^c}^{m_i}) \doteq E_{m_i}(v_k)/2$;
    Set $\mathbf{w}(v_{k^c}^{m_i} \rightarrow v_{k^t}^{m_i}) \doteq E_{m_i}(v_k)/2$;
  end
end
for $w_{i,j}$ *in* $\mathcal{W}_f$ **do**
  for $v_k^{m_i}$ *in* $\mathcal{V}_{m_i}$ **do**
    for $v_k^{m_j}$ *in* $\mathcal{V}_{m_j}$ **do**
      $\mathcal{V}_o^{re}, \mathcal{E}_o^{re}$ = **GISRSA**$(\mathcal{G}, v_k^{m_i}, v_k^{m_j})$;
      $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
      $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
    end
  end
end
for $v_k^{m_{src}}$ *in* $\mathcal{V}_{m_{src}}$ **do**
  $\mathcal{V}_o^{re}, \mathcal{E}_o^{re}$ = **GISRSA**$(\mathcal{G}, v^{src}, v_k^{m_{src}})$;
  $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
  $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
end
for $v_k^{m_{dst}}$ *in* $\mathcal{V}_{m_{dst}}$ **do**
  for g *in* $[1, \rho]$ **do**
    $\mathcal{V}_o^{re}, \mathcal{E}_o^{re}$ = **GISRSA**$(\mathcal{G}, v_k^{m_{dst}}, V^{dst_g})$;
    $\mathcal{V}_z \leftarrow \mathcal{V}_z \bigcup \mathcal{V}_o^{re}$;
    $\mathcal{E}_z \leftarrow \mathcal{E}_z \bigcup \mathcal{E}_o^{re}$;
  end
end

In $\mathcal{G}_f^m$, there may exist ρ $(1 \leq \rho \leq |\mathcal{M}_f| - 1)$ vertices whose outdegree equals 0. These vertices represent terminal microservices that conclude the processing sequence by dispatching results to their respective destinations. Consequently, there should be multiple logical destination satellite nodes, denoted as $\{v^{dst_1}, \ldots, v^{dst_\rho}\}$. The routing paths from the logical destination satellites hosting the terminal microservices to the real destination satellites can be straightforwardly generated by the control panel. Note that if a terminal microservice is deployed directly on its real destination satellite, this satellite also acts as the logical destination. In such cases, the communication cost between the terminal service and its destination is effectively zero. Next, we define the augmented graph to be constructed as $\mathcal{G}_z = (\mathcal{V}_z, \mathcal{E}_z)$, where initially, $\mathcal{V}_z = \varnothing$ and $\mathcal{E}_z = \varnothing$. Then, the process of constructing graph $\mathcal{G}_z$ is detailed as follows:

Step ❶. This step involves adding the source node v^{src}, along with the logical destination nodes $\{v^{dst_1}, \ldots, v^{dst_\rho}\}$ and the real destination node v_{dst}, to vertex collection $\mathcal{V}_z$.

Step ❷. For each microservice m_i in $\mathcal{M}_f$, and $\forall v_k \in \mathcal{V}_{m_i}$,

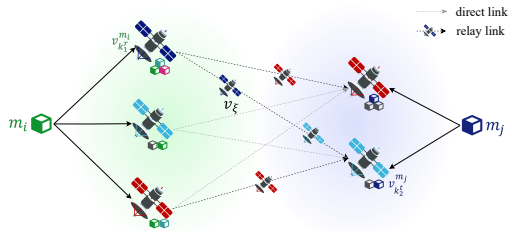


Fig. 3: Build the relay routing paths for neighbor microservices that are deployed on non-adjacent satellites.

this step creates three distinct vertices: $v_{k^r}^{m_i}$, $v_{k^c}^{m_i}$ and $v_{k^t}^{m_i}$, which are then added to the set $\mathcal{V}_z$. Subsequently, it constructs two directed edges $v_{k^r}^{m_i} \rightarrow v_{k^c}^{m_i}$ and $v_{k^c}^{m_i} \rightarrow v_{k^t}^{m_i}$, both with weights equal to $E_{m_i}(v_k)/2$. These edges are added to the edge set $\mathcal{E}_z$. This procedure is detailed in Fig. 2.

Step ❸. For the calling relationship between two microservices m_i and m_j, that is, $\forall w_{i,j} \in \mathcal{W}_f$, the algorithm will iteratively identify all satellite nodes from collection $\mathcal{V}$ that host microservices m_i and m_j, respectively. If satellites $v_k^{m_i}$ and $v_k^{m_j}$ are directly adjacent without needing any relay satellite to facilitate the communication, then a directed edge $v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}$ is constructed, and its weight is set as the communication energy cost from $v_{k^r}^{m_i}$ to $v_{k_t}^{m_j}$ in $\mathcal{G}$, given as:

$$\mathbf{e}^{tr} = P_T^{v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}} \frac{w_{i,j}}{\mathrm{w}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})}, \tag{23}$$

Otherwise, the algorithm then searches for available relay satellite nodes within $\mathcal{V}$, which can be efficiently resolved using the Floyd algorithm, resulting in a collection of relay satellite nodes denoted as $\mathcal{V}^{re} = \{v_1^{re}, \ldots v_\xi^{re}\}$. Then, it adds these relay nodes to $\mathcal{V}_z$, and constructs and adds edges $v_{k^t}^{m_i} \rightarrow v_a^{re}, v_a^{re} \rightarrow v_{a+1}^{re}, v_\xi^{re} \rightarrow v_{k^r}^{m_j}, (\forall a \in [1, \xi - 1])$ to $\mathcal{E}_z$. The weights of these edges, representing the relay communication energy cost between satellites, are calculated according to Eq. **??** and **??**, as below:

$$\mathbf{e}_{re_1}^{tr}(v_{k^t}^{m_i} \rightarrow v_a^{re}) = P_T^{v_{k^t}^{m_i} \rightarrow v_a^{re}} \frac{w_{i,j}}{\mathrm{w}(v_{k^t}^{m_i} \rightarrow v_a^{re})}, \tag{24}$$

$$\mathbf{e}_{re_2}^{tr}(v_a^{re} \rightarrow v_{a+1}^{re}) = P_T^{v_a^{re} \rightarrow v_{a+1}^{re}} \frac{w_{i,j}}{\mathrm{w}(v_a^{re} \rightarrow v_{a+1}^{re})}, \tag{25}$$

$$\mathbf{e}_{re_3}^{tr}(v_\xi^{re} \rightarrow v_{k^r}^{m_j}) = P_T^{v_\xi^{re} \rightarrow v_{k^r}^{m_j}} \frac{w_{i,j}}{\mathrm{w}(v_\xi^{re} \rightarrow v_{k^r}^{m_j})}. \tag{26}$$

This process is depicted in Fig . 3.

Step ❹. For the first executed microservice m_{src} and the final executed microservices $\{m_{dst^1}, \ldots, m_{dst^\rho}\}$, the algorithm searches for the collection of satellite nodes hosting these microservices. Subsequently, it constructs the edges $v^{src} \rightarrow v_{k^r}^{m_{src}}$ and $v_{k^t}^{m_{dst}} \rightarrow v^{dst_g}, \forall g \in [1, \rho]$. The detailed procedures for the construction of the newly generated vertices and edges are similar to those outlined in **Step ❸**.

We summarize the above steps in Algorithm 1. This systematic approach guarantees that within the augmented graph $\mathcal{G}_z$, each tree rooted at the source node v^{src}, which includes the destination node v^{dst} and all $v_{k^c}^{m_i}$ nodes, constitutes a feasible service routing scheme ψ. This scheme ensures that

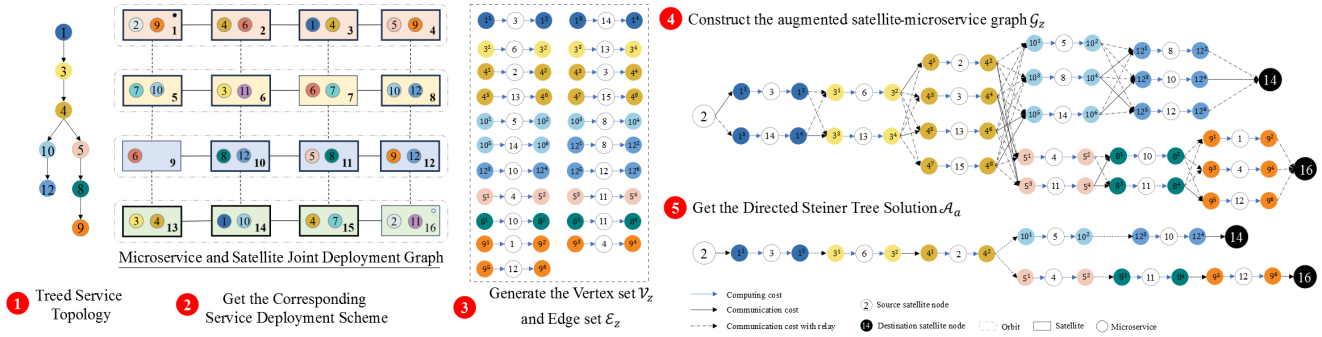


Fig. 4: The overview of solution workflow of tree-based request topology.

all computation and communication processes required in $\mathcal{G}_f^m$ are executed sequentially, and the sum of the weights of all edges in this tree quantifies the total energy E^{total} required for the routing scheme. Subsequently, solving the original problem can essentially be transformed into tackling a DST problem on the graph $\mathcal{G}_z$. The objective is to identify a connected subgraph that contains all specified nodes in graph $\mathcal{G}_z$ such that the sum of the weights of all edges in the subgraph is minimized. The directed Steiner tree problem is defined as follows.

Definition 4. (Directed Steiner Tree). *Given a directed graph G=(V, A), an assigned root node $r \in V$, and a set of terminals $X \subseteq V, (|X| = k)$, the objective is to find the minimum cost arborescence rooted at r and spanning all the vertices in X.*

Algorithm 2: Approximated Solution for Minimized Spanning Directed Steiner Arborescence.

Input : $\mathcal{G}_z, \{v^{src}, v^{dst_1}, \dots v^{dst_\rho}\}$
Output: $\mathcal{A}(\mathcal{V}_\mathrm{a}, \mathcal{E}_\mathrm{a})$

Init the terminal pairs set, $\{(v^{src}, v^{dst_\mathrm{g}}) | \forall \mathrm{g} \in [1, \rho]\}$;
Init $\mathcal{V}_\mathrm{a} \leftarrow \varnothing, \mathcal{E}_\mathrm{a} \leftarrow \varnothing$;
for *g in* $[1, \rho]$ **do**
  $\mathcal{V}_g, \mathcal{E}_g$ = Dijkstra($\mathcal{G}_z, v^{src}, v^{dst_\mathrm{g}}$)
end
$\mathcal{V}_\mathrm{a} \leftarrow \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{V}_\mathrm{g}$;
$\mathcal{E}_\mathrm{a} \leftarrow \mathcal{E}_1$;
for *g in* $[2, \rho]$ **do**
  for *e in* $\mathcal{E}_g$ **do**
    if $e \in \mathcal{E}_a$ **then**
      $\mathrm{w}(e) \doteq \min(\mathrm{w}(\mathcal{E}_\mathrm{a}^e), \mathrm{w}(\mathcal{E}_\mathrm{g}^e))$
    else
      $\mathcal{E}_\mathrm{a} \leftarrow \mathcal{E}_\mathrm{a} \cup \{e\}$
    end
  end
end
Construct $\mathcal{A}(\mathcal{V}_\mathrm{a}, \mathcal{E}_\mathrm{a})$;

B. DST-based Service Routing Algorithm

The DST problem is known to be NP-hard, and extensive research has been conducted on it. Numerous studies have proposed approximation solutions for the original problem [32], [33]. However, solving the DST problem within the LEO network topology introduces unique and particularly severe challenges due to the dynamic nature of satellite constellations. Specifically, the time-varying nature of ISLs owing to the constant movement of satellites results in a highly dynamic network topology, making the link connectivity a complex issue. LoS constraints further complicate the ability to guarantee consistent connections between terminals. Additionally, microservices that need to communicate may not be colocated on the same or directly adjacent satellites, necessitating the use of relay satellites to establish communication paths. These challenges motivate us to design a specialized heuristic algorithm tailored to the ever-changing LEO satellite networks.

Algorithm 3: Generate the Inter-Satellite Relay Routing Scheme Algorithm (**GISRA**).

Input : $\mathcal{G}, v_k^{m_i}, v_k^{m_j}$
Output: $\mathcal{V}_o^{re}, \mathcal{E}_o^{re}$

Initialize the vertex set and edge set with $\mathcal{V}_o^{re} \leftarrow \varnothing, \mathcal{E}_o^{re} \leftarrow \varnothing$;
if $v_k^{m_i}$ *is adjacent to* $v_k^{m_j}$ **then**
  $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_z \bigcup \{(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})\}$;
  $\mathbf{w}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j}) \doteq \mathbf{e}^{tr}(v_{k^r}^{m_i} \rightarrow v_{k^t}^{m_j})$;
else
  $\mathcal{V}_o^{re}$, path = Floyd($\mathcal{G}, v_k^{m_i}, v_k^{m_j}$) ;
  $\mathcal{V}_o^{re} \leftarrow \mathcal{V}_o^{re} \setminus \{v_k^{m_i}, v_k^{m_i}\}$;
  $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_o^{re} \bigcup \{(v_{k^t}^{m_i} \rightarrow v_1^{re}), (v_\xi^{re} \rightarrow v_{k^r}^{m_j})\}$;
  $\mathbf{w}(v_{k^t}^{m_i} \rightarrow v_1^{re}) \doteq \mathbf{e}_{re_1}^{tr}(v_{k^t}^{m_i} \rightarrow v_1^{re})$;
  $\mathbf{w}(v_\xi^{re} \rightarrow v_{k^r}^{m_j}) \doteq \mathbf{e}_{re_3}^{tr}(v_\xi^{re} \rightarrow v_{k^r}^{m_j})$;
  for $a \in [1, \xi - 1]$ **do**
    $\mathcal{V}_o^{re} \leftarrow \mathcal{V}_o^{re} \bigcup \{v_a^{re}, v_{a_1}^{re}\}$;
    $\mathcal{E}_o^{re} \leftarrow \mathcal{E}_o^{re} \bigcup \{(v_a^{re} \rightarrow v_{a_1}^{re})\}$;
    $\mathbf{w}(v_a^{re} \rightarrow v_{a_1}^{re}) \doteq \mathbf{e}_{re_2}^{tr}(v_a^{re} \rightarrow v_{a_1}^{re})$;
  end
end

The main steps of our energy-aware DST-based service routing algorithm are as follows:

1) Augmented Graph Initialization: In time slot s, the augmented graph $\mathcal{G}_z(\mathrm{s}) = (\mathcal{V}_z(\mathrm{s}), \mathcal{E}_z(\mathrm{s}))$ is initialized with weighed edges.

2) Get the Shortest Path: For each root and terminal node pairs $(v^{src}, v^{dst_\mathrm{g}}), \forall \mathrm{g} \in [1, \rho]$, it recursively finds the

minimum cost path by utilizing the Dijkstra algorithm. Record the involved vertices and edges that constitute each path, represented by $\{\mathcal{V}_1(\mathrm{s}), \ldots, \mathcal{V}_\rho(\mathrm{s})\}$ and $\{\mathcal{E}_1(\mathrm{s}), \ldots, \mathcal{E}_\rho(\mathrm{s})\}$.

3) Merge the Shortest Paths: Aggregate the vertex sets from the minimum cost paths and remove any duplicates to obtain $\mathcal{V}_\mathrm{a}(\mathrm{s}) = \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{V}_\mathrm{g}(\mathrm{s})$. Then, consolidate these edge sets to $\mathcal{E}_\mathrm{a}(\mathrm{s}) = \bigcap_{\mathrm{g}=1}^{\rho} \mathcal{E}_\mathrm{g}(\mathrm{s})$. Note that if any edges share common vertices, select the edge with the smaller weight to ensure the most cost-effective connections.

4) Minimum Spanning Steiner Arborescence: Utilize the aggregated vertex and edge collections to construct the minimum spanning Steiner arborescence for the snapshot s, denoted by $\mathcal{A}(\mathrm{s}) = (\mathcal{V}_\mathrm{a}(\mathrm{s}), \mathcal{E}_\mathrm{a}(\mathrm{s}))$. This step provides an approximate solution to the DST problem, optimizing the cost while ensuring connectivity among specified nodes.

Algorithm 2 presents a detailed step-by-step procedure for the proposed approach, offering a comprehensive view of its implementation. To enhance understanding, a simplified example is provided in Fig. 4, which visually demonstrates the key steps and logic of the algorithm.

C. Algorithm Complexity Analysis

The complexity of Algorithm 1 and 2 is analyzed as follows.

Complexity of Algorithm 1. The time complexity of Dijkstra algorithm is given by $\mathcal{O}(|\mathcal{E}| + |\mathcal{V}| \log |\mathcal{V}|)$. In the worst case, the algorithm needs to run $|\mathcal{V}^*|$ times, which is the number of destination satellites, and the total time complexity deteriorates to $\mathcal{O}(|\mathcal{V}^*|(|\mathcal{E}| + |\mathcal{V}| \log |\mathcal{V}|))$. This signifies that the complexity is contingent upon the structure of both the service request graph and the space network topology.

Complexity of Algorithm 2. The time complexity of merging these distinct sets into one set is $\mathcal{O}(\mathrm{n})$, where n is the total number of elements across all sets. Therefore, the time complexity of obtaining the minimum DST is inferred as $\mathcal{O}(|\mathcal{V}^*|(|\mathcal{E}|+|\mathcal{V}| \log |\mathcal{V}|)+2{*}\mathrm{n}+(|\mathcal{V}_z|+|\mathcal{E}_z|))$, where $\mathcal{O}(2{*}\mathrm{n})$ represents the time complexity of merging the generated vertex and edge sets, and $\mathcal{O}(|\mathcal{V}_z| + |\mathcal{E}_z|)$ is the time complexity of constructing the directed Steiner tree.

VI. Performance Evaluation

A. Experiment Setup

Constellation. In experiments, we consider a 72/12/1 Telesat Star Constellation system, which consists of $P = 6$ polar orbit planes whose inclination is 99.5°, and the orbital altitude is $h = 1000km$ on the ground [34].

Satellite Computing Capability. We suppose LEO satellites are equipped with several onboard processors and storage components. According to [19], we assign the satellite computing capability following a uniform distribution, as $\varphi(v_k) \sim \mathrm{DiscreteUniform}(\{100, 160, 220, 280, 340, 400\})$, while the processor power $\Psi(v_k) = \varphi(v_k) \times 0.01$ in Watts.

ISL Parameters. Similar to settings in [29], [30], [35], the transmission power of all relevant satellites is standardized at $P_T = 3$ (Watt). In the context of the Telesat Constellation, which achieve inter-satellite interconnection across free-space optical link, the carrier frequency of each of satellite is set as f_c=193(THz), , and the corresponding carrier wavelength is λ=1550(nm) while the channel bandwidth between two satellites is defined as BW=2%f_c. Besides, we specify the antenna diameter transceiver as D_R=6(mm), the pointing loss angle $\theta_0 = 0.01\pi$ and the optical conversion efficiency of the transceiver modules $\eta = 0.8$.

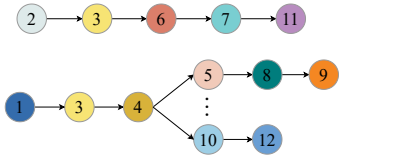
Microservices. According to the microservice setting adopted by [19], we heuristically construct the microservices testbed as follows: There are $|\mathcal{M}| = 60$ microservices whose data volume follows a Gaussian distribution with a mean of 50 and variance of 4 in Gbits. For each microservice, there are $4 \sim 6$ replicas distributed on satellites, while each satellite can concurrently host at most $\lfloor |\mathcal{M}| \times 6/72 \rfloor$ microservices.

Ablation Solutions: We design the following benchmarks to evaluate the performance of our proposed algorithm.

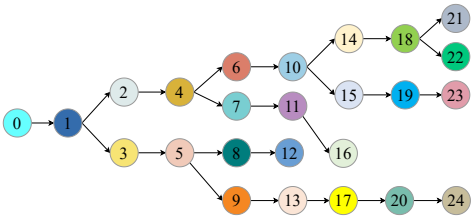
1) **Random:** It randomly selects a satellite for each microservice and uses the Dijkstra algorithm to get the shortest path when dealing with relay links.
2) **Computation greedy (Comp-greedy):** It chooses the satellite that offers the minimum computation energy cost for executing each microservice. The relay routing policy is the same as **Random** policy.
3) **Communication greedy (Comm-greedy):** It picks the satellite pair with the minimum communication energy cost, including the potential relay links.

We define three service request topologies: the Chained topology, the multi-forked Treed topology, and the Sophisticated Treed (Soph Tree) topology, as depicted in Fig. 5.

To verify the effectiveness of our algorithm, we conducted a series of experiments on a space topology comprising 72 satellites, each hosting 60 microservices with 4∼6 replicas. We generated 100 service request chains, varying in length from 10 to 20, to simulate a chained microservice topology. We also constructed 100 random treed topologies, containing 20 to 45, featuring a stochastic number of leaf nodes and varying branch configurations. For each sophisticated treed topology, microservices were randomly assigned to nodes,



(a) Chained and Treed topologies.



(b) Sophisticated-Treed (Soph-Treed) topology.

Fig. 5: The overview of service request topologies from terrestrial stations.

and this assignment process was independently repeated 100 times. For each topology, energy cost data was collected from 100 samples. The final results were derived by averaging the energy cost values across these samples. It ensures robust and reliable performance evaluation by mitigating variations caused by randomness in the topology generation and microservice allocation processes.

B. Experimental Results

1) Effectiveness: As illustrated in Fig. 6(a), our proposed Steiner consistently outperforms all baseline methods across various microservices topology scenarios, and its advantages become more pronounced in more complex topologies. This enhanced performance is charged to our comprehensive approach that simultaneously considers both computation and communication costs when determining the most efficient routing paths. The comp-greedy method outperforms the random method by prioritizing minimizing computation costs. The comm-greedy method surpasses both comp-greedy and random methods by focusing on minimizing communication costs. These findings emphasize the necessity of accounting for both computation and communication factors to design effective and energy-conscious routing strategies for the SCPN.

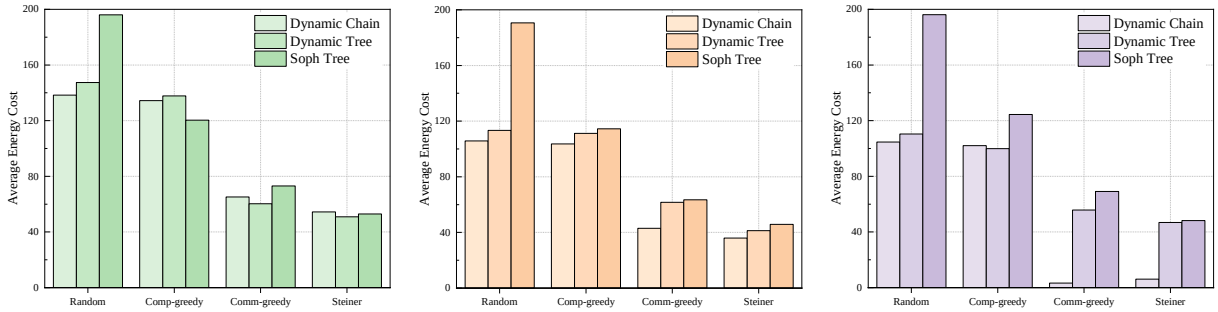
2) Impact of constellation scales: To evaluate the scalability of our algorithm, we systematically varied the number of satellites in the constellation by adjusting the number of orbits while maintaining consistent microservice deployment strategies. Experiments were conducted in two constellation settings separately, as illustrated in Fig. 6(b) and Fig. 6(c). These configurations allowed us to assess the algorithm's performance under varying levels of network complexity and service topology. Our results demonstrate that the proposed approach consistently outperforms alternative methods across all tested constellation scales and topological scenarios. Notably, its effectiveness becomes increasingly pronounced as the network complexity escalates, highlighting its robustness and adaptability to intricate environments.

3) Impact of transmitter power: Given the dominant role of communication costs in the total energy consumption of satellite systems, we conducted a series of experiments to explore how varying channel conditions influence the performance of our Steiner scheme. Specifically, we experimented with four different transmitter power levels on satellites across diverse microservice request schemes to examine their effect on energy efficiency. The results shown in Figs. 7(a), 7(b) and 7(c) indicate that as transmitter power increases, the average energy cost shows an upward trend. This trend highlights the trade-off between signal strength and energy consumption, with higher power levels leading to greater energy expenditure. Despite this increase in energy cost, our Steiner algorithm consistently outperformed other benchmark strategies across all service request scenarios. This superior performance is attributed to the algorithm's ability to dynamically adapt to the complexities of space network topologies and fluctuating channel conditions. By optimizing service routing paths, our approach minimizes unnecessary energy use under varying channel conditions. These findings underscore the importance of intelligent service routing mechanisms in the SCPN, particularly in environments with variable communication links.

4) Impact of faulty satellites: Given the constraints of limited power supply and the effects of solar eclipses, some satellites may need to enter a low-power mode, functioning solely as relay nodes for message forwarding [17]. To evaluate the robustness of our proposed method under such conditions, we conducted experiments by randomly disabling a subset of satellites in a constellation of 72 satellites. Specifically, we tested scenarios with 10, 20, and 25 faulty satellites by eliminating their microservices, as shown in Figs 8(a), 8(b), 8(c). The results reveal that energy cost generally increases with an increasing number of faulty satellites for all methods. This trend can be attributed to the reduced availability of active nodes, which forces longer transmission paths and higher energy consumption for maintaining connectivity (much more relay satellites are required for message forwarding). Despite this challenge, Steiner invariably outperforms other methods in all scenarios. Its superior performance is attributed to its robust design, which can effectively adapt to changes in network topology and operational conditions. By optimizing power allocation and service routing decisions, the algorithm minimizes disruptions caused by satellite failures.

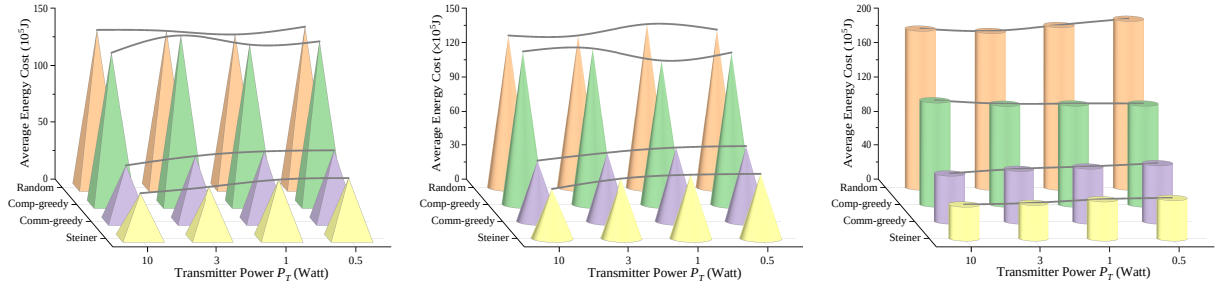
5) Impact of microservice deployment schemes: To investigate the influence of microservice deployment and scaling strategies on energy consumption with in the SCPN, we conducted experiments by varying the number of replicas allocated for each microservice. Specifically, we tested three replica ranges: $[2, 6]$, $[4, 6]$, and $[6, 10]$. And the experimental results, illustrated in Figs. 9(a), 9(b), and 9(c), reveal that energy costs generally decreases as the scale of microservice deployment expands. This trend is attributed to the increased availability of service nodes and routing paths, which provide more opportunities for route optimization and efficient power utilization. Our algorithm regularly demonstrates superior performance in reducing energy costs compared to other benchmarks, particularly in more complex microservice-satellite patterns. This superior performance can be attributed to the algorithm's ability to leverage the increased number of service routing paths and nodes, enabling more advanced route strategies. The expanded deployment range provides greater flexibility in service selections and routing decisions, allowing the algorithm to identify more energy-efficient configurations. By contrast, the energy costs associated with routing paths generated by random and comp-greedy methods occasionally increased as the number of replicas grew. This counterintuitive behavior can be attributed to inefficient routing decisions caused by the higher number of service nodes and calling paths, which complicate the decision-making process and lead to suboptimal resource utilization.

6) Adaptability and stability: Furthermore, as depicted in Fig 10, which presents the energy distribution derived from 300 independent experiments, underscores the superior stability of the Steiner policy compared to other benchmark strategies. The results reveal that, even under sophisticated



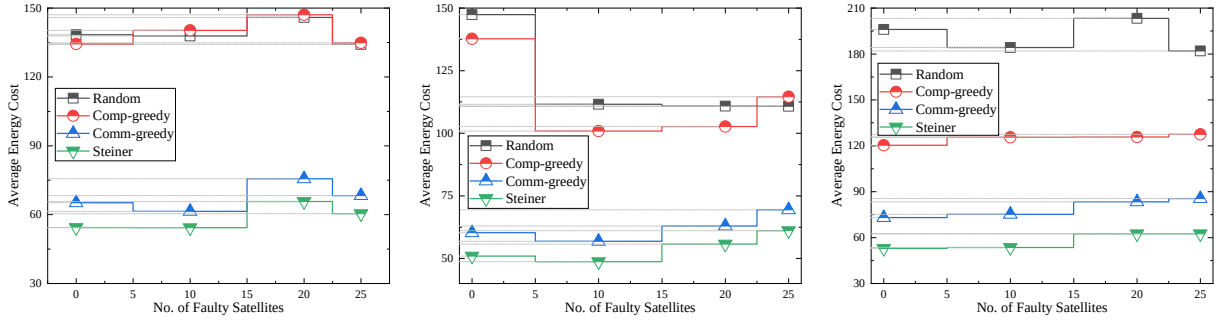
(a) Constellation with 72 satellites. (b) Constellation with 48 satellites. (c) Constellation with 108 satellites.

Fig. 6: Performance comparison on energy-cost under varying constellation scales for dynamic microservice request topologies.



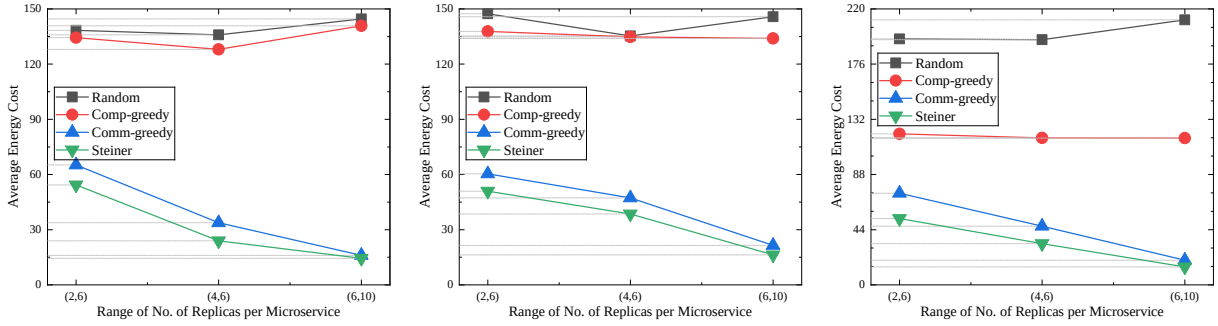
(a) Chained service topology. (b) Treed service topology. (c) Sophisticated Treed service topology.

Fig. 7: Performance comparison on energy-cost under different onboard transmitter power configurations for dynamic microservice request topologies.



(a) Robustness over faulty satellites (Chain). (b) Robustness over faulty satellites (Tree). (c) Robustness over faulty satellites (Soph Tree).

Fig. 8: Performance comparison on energy-cost under different configurations on number of faulty satellites for dynamic service request topologies.



(a) Microservice scale (Chain). (b) Microservice scale (Tree). (c) Microservice scale (Soph Tree).

Fig. 9: Performance comparison on energy-cost under different configurations on number of replicas of microservices for dynamic service request topologies.

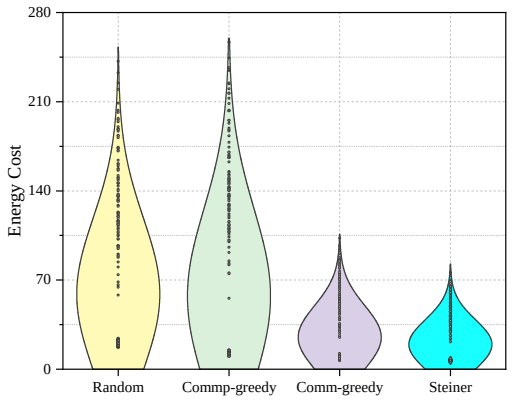


Fig. 10: Energy distribution of Soph Tree.

treed topologies with heightened complexity, the Steiner policy exhibits less variance in energy costs. This stability highlights its robustness and adaptability in managing diverse network conditions and service requirements.

VII. Conclusion

In this paper, we proposed a DST-based solution to provide an energy-efficient service routing scheme for tasks from terrestrial clients. We systematically developed the SCPN models, including communication and computation, covering various aspects of the LEO satellite network. We also mathematically formulated the energy optimization problem for satellite-microservice routing and transformed it into finding a feasible DST solution within an augmented directed satellite-microservice colocated graph. To approximate the optimal solution, we designed a series of heuristic algorithms. Experimental results show that our algorithm significantly outperforms other benchmarks in energy efficiency, robustness, and adaptability across different network and microservice configurations in the context of SCPN.

References

[1] Z. Xiao, J. Yang, T. Mao, C. Xu, R. Zhang, Z. Han, and X.-G. Xia, "Leo satellite access network (leo-san) towards 6g: Challenges and approaches," *IEEE Wireless Communications*, 2022.

[2] S. Ma, Y. C. Chou, H. Zhao, L. Chen, X. Ma, and J. Liu, "Network characteristics of leo satellite constellations: A starlink-based measurement from end users," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pp. 1–10, 2023.

[3] H. Al-Hraishawi, H. Chougrani, S. Kisseleff, E. Lagunas, and S. Chatzinotas, "A survey on nongeostationary satellite systems: The communication perspective," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 101–132, 2023.

[4] B. Zhang, Y. Wu, B. Zhao, J. Chanussot, D. Hong, J. Yao, and L. Gao, "Progress and challenges in intelligent remote sensing satellite systems," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 1814–1822, 2022.

[5] Y. Li, M. Wang, K. Hwang, Z. Li, and T. Ji, "Leo satellite constellation for global-scale remote sensing with on-orbit cloud AI computing," *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, vol. 16, pp. 9369 – 9381, 2023.

[6] Z. Zhai, L. Zeng, T. Ouyang, S. Yu, Q. Huang, and X. Chen, "Seco: Multi-satellite edge computing enabled wide-area and real-time earth observation missions," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pp. 2548–2557, 2024.

[7] I. Leyva-Mayorga, M. Martinez-Gost, M. Moretti, A. Pérez-Neira, M. Á. Vázquez, P. Popovski, and B. Soret, "Satellite edge computing for real-time and very-high resolution earth observation," *IEEE Transactions on Communications*, vol. 71, no. 10, pp. 6180–6194, 2023.

[8] Q. Ouyang, N. Ye, J. Gao, A. Wang, and L. Zhao, "Joint in-orbit computation and communication for minimizing download time from leo satellites," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3950–3963, 2024.

[9] Z. Shen, J. Jin, C. Tan, A. Tagami, S. Wang, Q. Li, Q. Zheng, and J. Yuan, "A survey of next-generation computing technologies in space-air-ground integrated networks," *ACM Comput. Surv.*, vol. 56, aug 2023.

[10] B. Shang, Y. Yi, and L. Liu, "Computing over space-air-ground integrated networks: Challenges and opportunities," *IEEE Network*, vol. 35, no. 4, pp. 302–309, 2021.

[11] C. Wang, Y. Zhang, Q. Li, A. Zhou, and S. Wang, "Satellite computing: A case study of cloud-native satellites," in *2023 IEEE International Conference on Edge Computing and Communications (EDGE)*, pp. 262–270, 2023.

[12] S. Ye, J. Du, L. Zeng, W. Ou, X. Chu, Y. Lu, and X. Chen, "Galaxy: A resource-efficient collaborative edge ai system for in-situ transformer inference," *arXiv preprint arXiv:2405.17245*, 2024.

[13] D. Merkel *et al.*, "Docker: lightweight linux containers for consistent development and deployment," *Linux j*, vol. 239, no. 2, p. 2, 2014.

[14] T. Salah, M. J. Zemerly, C. Y. Yeun, M. Al-Qutayri, and Y. Al-Hammadi, "The evolution of distributed systems towards microservices architecture," in *2016 11th International Conference for Internet Technology and Secured Transactions (ICITST)*, pp. 318–325, IEEE, 2016.

[15] J. Wang, "Towards geoai as a containerized microservice," in *Proceedings of the 31st ACM International Conference on Advances in Geographic Information Systems*, pp. 1–2, 2023.

[16] A. Ruiz-Zafra, J. Pigueiras-del Real, M. Noguera, L. Chung, D. G. Barres, and K. Benghazi, "Servitization of customized 3d assets and performance comparison of services and microservices implementations," *IEEE Transactions on Services Computing*, vol. 17, no. 1, pp. 194–208, 2024.

[17] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3869–3886, 2016.

[18] Y. Zhang, Q. Wu, Z. Lai, Y. Deng, H. Li, Y. Li, and J. Liu, "Energy drain attack in satellite internet constellations," in *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pp. 1–10, 2023.

[19] J. Cao, S. Zhang, Q. Chen, H. Wang, M. Wang, and N. Liu, "Computing-aware routing for leo satellite networks: A transmission and computation integration approach," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 12, pp. 16607–16623, 2023.

[20] Y. Gao, Z. Yan, K. Zhao, T. de Cola, and W. Li, "Joint optimization of server and service selection in satellite-terrestrial integrated edge computing networks," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 2, pp. 2740–2754, 2024.

[21] Q. Xia, G. Wang, Z. Xu, W. Liang, and Z. Xu, "Efficient algorithms for service chaining in nfv-enabled satellite edge networks," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 5677–5694, 2024.

[22] H. G. Abreha, H. Chougrani, I. Maity, Y. Drif, C. Politis, and S. Chatzinotas, "Fairness-aware vnf mapping and scheduling in satellite edge networks for mission-critical applications," *IEEE Transactions on Network and Service Management*, vol. 21, no. 6, pp. 6716–6730, 2024.

[23] X. He, T. Wang, Y. Shi, and X. Liu, "Throughput-optimized service routing for microservice flows in leo satellite networks," *IEEE Transactions on Mobile Computing*, vol. 25, no. 5, pp. 7196–7209, 2026.

[24] Z. Yu, H. V. Cheng, Z. Yang, X. Liu, Y. Jiang, Y. Zhou, and Y. Shi, "Microservice deployment for satellite edge ai inference via deep reinforcement learning," in *2024 IEEE 35th International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, pp. 1–6, 2024.

[25] D. Ron, F. A. Yusufzai, S. Kwakye, S. Roy, N. Sastry, and V. K. Shah, "Time- dependent network topology optimization for leo satellite constellations," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pp. 1–10, 2025.

[26] Y. Yu, J. Yang, C. Guo, H. Zheng, and J. He, "Joint optimization of service request routing and instance placement in the microservice system," *Journal of Network and Computer Applications*, vol. 147, p. 102441, 2019.

[27] K. Peng, L. Wang, J. He, C. Cai, and M. Hu, "Joint optimization of service deployment and request routing for microservices in mobile edge

computing," *IEEE Transactions on Services Computing*, vol. 17, no. 3, pp. 1016–1028, 2024.

[28] N. Cheng, F. Lyu, W. Quan, C. Zhou, H. He, W. Shi, and X. Shen, "Space/aerial-assisted computing offloading for iot applications: A learning-based approach," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 5, pp. 1117–1129, 2019.

[29] I. Leyva-Mayorga, B. Soret, and P. Popovski, "Inter-plane inter-satellite connectivity in dense leo constellations," *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3430–3443, 2021.

[30] A. Polishuk and S. Arnon, "Optimization of a laser satellite communication system with an optical preamplifier," *J. Opt. Soc. Am. A*, vol. 21, pp. 1307–1315, Jul 2004.

[31] A. Fu, E. Modiano, and J. Tsitsiklis, "'optimal energy allocation and admission control for communications satellites," *IEEE/ACM Transactions on Networking*, vol. 11, no. 3, pp. 488–500, 2003.

[32] M. Charikar, C. Chekuri, T. yat Cheung, Z. Dai, A. Goel, S. Guha, and M. Li, "Approximation algorithms for directed steiner problems," *Journal of Algorithms*, vol. 33, no. 1, pp. 73–91, 1999.

[33] M. Medard, S. Finn, R. Barry, and R. Gallager, "Redundant trees for preplanned recovery in arbitrary vertex-redundant or edge-redundant graphs," *IEEE/ACM Transactions on Networking*, vol. 7, no. 5, pp. 641–652, 1999.

[34] I. Del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three low earth orbit satellite constellation systems to provide global broadband," *Acta astronautica*, vol. 159, pp. 123–135, 2019.

[35] S. Nie and I. F. Akyildiz, "Channel modeling and analysis of inter-small-satellite links in terahertz band space networks," *IEEE Transactions on Communications*, vol. 69, no. 12, pp. 8585–8599, 2021.